Universidad Nacional de Costa Rica.

Facultad de ciencias exactas y naturales.

Documentación del Primer Proyecto de Sistemas Operativos.

Integrantes:

Jarod Bonilla.

Luis Hidalgo.

Gabriel Quiros.

Profesor: José Calvo Suarez.

Sistemas Operativos

3 de mayo del 2026.

Heredia, Costa Rica.

1. Descripción General del Proyecto

Este proyecto implementa un simulador de terminal logística en lenguaje C, usando la librería POSIX de hilos (pthreads). El objetivo principal es modelar un escenario donde múltiples camiones representados como hilos compiten por acceder a un recurso limitado: los muelles de carga (sección crítica).

El sistema está compuesto por 8 camiones y 3 muelles disponibles simultáneamente. Cada camión tiene un tiempo de carga (burst) y una prioridad (1 = perecedero / alta prioridad, 2 = normal). El servidor de la terminal es el proceso principal que gestiona todo mediante algoritmos de planificación.

2. Arquitectura del Sistema

2.1 Módulos implementados

- terminal.h : Definiciones globales: constantes, structs y enums compartidos por todos los módulos.
- camion.c / camion.h : Lógica de cada hilo-camión. Gestiona el ciclo de vida completo.
- scheduler.c / scheduler.h: Cola de planificación. Implementa FIFO y Round Robin con prioridad.
- sincronizacion.c / sincronizacion.h : Inicializa el semáforo de muelles y el mutex del log.
- logger.c / logger.h : Registro de eventos. Protegido con mutex para evitar condiciones de carrera.
- main.c : Proceso principal. Lanza ambas simulaciones y genera la tabla comparativa final.

2.2 Ciclo de vida de cada hilo-camión

Cada hilo pasa por los siguientes estados, implementados en camion.c:

Estado	Cuando ocurre	Implementación en el código
NUEVO	Al crear el hilo con pthread_create()	c->estado = ESTADO_NUEVO al inicio de hilo_camion()
LISTO	Entra a la cola del scheduler	scheduler_encolar(c) + c->estado = ESTADO_LISTO
BLOQUEADO	Esperando sem_wait() o mutex_log	sem_wait(&sem_muelles) bloquea el hilo hasta muelle libre
EJECUCIÓN	Dentro de la sección crítica (muelle)	sleep(burst) o ciclo de quantums según algoritmo
TERMINADO	Carga completada, muelle liberado	sem_post + c->estado = ESTADO_TERMINADO + pthread_join

3. Sincronización: Semáforos y Mutex

3.1 Semáforo de muelles (sem_muelles)

Se inicializa con valor 3 (NUM_MUELLES). Esto garantiza que como máximo 3 camiones accedan simultáneamente al muelle. Cuando los 3 están ocupados, el hilo que intenta sem_wait() queda bloqueado por el sistema operativo hasta que otro haga sem_post().

```
sem_init(&sem_muelles, 0, NUM_MUELLES);   // sincronizacion.c

sem_wait(&sem_muelles);                   // bloquea si no hay muelle libre

sem_post(&sem_muelles);                   // libera el muelle al terminar
```

3.2 Mutex del Log (mutex_log)

Múltiples hilos pueden intentar escribir en terminal.log al mismo tiempo. Para evitar que los mensajes se mezclen o corrompan (race condition), cada escritura está protegida por un mutex:

```
pthread_mutex_lock(&mutex_log);

fprintf(archivo_log, "C%-5d %-12s ...", camion_id, estado, ...);

pthread_mutex_unlock(&mutex_log);
```

3.3 Mutex de la cola del scheduler (mutex_cola)

La cola de planificación es accedida por múltiples hilos simultáneamente (para encolarse y descolarse). El scheduler usa su propio mutex interno (mutex_cola) para proteger las operaciones de inserción y extracción, evitando condiciones de carrera en la estructura de datos.

4. Algoritmos de Planificación

4.1 FIFO (First-In, First-Out)

Los camiones son atendidos estrictamente en el orden en que llegan a la cola. No existe desalojo: una vez que un camión ocupa un muelle, lo mantiene hasta terminar su carga completa (sleep(burst_restante)). Es simple y predecible, pero puede generar el efecto convoy: si un camión con burst largo llega primero, todos los demás esperan.

4.2 Round Robin con Prioridad

Cada camión ocupa el muelle por un máximo de QUANTUM = 2 segundos. Si no termina su carga, libera el muelle voluntariamente y regresa al final de la cola. Además, la cola está ordenada por prioridad: los camiones con carga perecedera (prioridad 1) siempre van adelante de los normales (prioridad 2). Esto se implementa mediante un ordenamiento burbuja al insertar en la cola en scheduler.c.

5. Prevención de Deadlock

¿Qué es un deadlock?

Un deadlock es cuando dos o más hilos se quedan esperando para siempre porque cada uno tiene un recurso que el otro necesita, y ninguno lo suelta. Por ejemplo: hilo A tiene el mutex y espera el semáforo; hilo B tiene el semáforo y espera el mutex. Resultado: bloqueo eterno.

¿Cómo lo evitamos en este proyecto?

La estrategia principal es simple: cada hilo nunca toma más de un recurso a la vez. Esto rompe la condición de "espera circular" que necesita un deadlock para formarse.

Proceso paso a paso:

- Cuando el camión quiere entrar al muelle, hace sem_wait(). En ese momento NO tiene el mutex del log ni ningún otro recurso tomado. Solo espera el semáforo.
- Cuando el camión quiere escribir en el log, hace mutex_lock(). En ese momento ya está DENTRO del muelle, pero el mutex del log lo toma y lo suelta casi inmediatamente (solo dura lo que tarda un fprintf). No hay forma de que se quede trancado ahí esperando algo más.
- El mutex del scheduler (mutex_cola) también se toma y se suelta en operaciones cortísimas (encolar/desencolar). Nunca coincide con el semáforo ni el mutex del log al mismo tiempo.

En términos técnicos: se cumple la condición de "orden de adquisición consistente" y no hay "retención con espera" prolongada. Cada hilo sigue este flujo lineal sin retrocesos:

sem_wait(muelle) → [ejecuta carga] → sem_post(muelle)

mutex_lock(log) → fprintf() → mutex_unlock(log) [dentro de la carga]

Como ningún hilo espera un recurso mientras sostiene otro de diferente tipo, el deadlock es estructuralmente imposible en este diseño.

6. Análisis Comparativo de Algoritmos

6.1 Resultados por camión FIFO

Camión	Prioridad	Burst (s)	T. Espera (s)	T. Retorno (s)	Observación
C1	Normal	5	0.00	5.00	Llega primero, entra directo
C2	Perecedero	3	0.00	3.00	Llega primero, entra directo
C3	Normal	4	0.00	4.00	Llega primero, entra directo
C4	Perecedero	2	3.00	5.00	Espera a que C2 libere muelle
C5	Normal	6	4.00	10.00	Espera a que C3 libere muelle
C6	Perecedero	1	5.00	6.00	Espera a que C1 libere muelle
C7	Normal	3	5.00	8.00	Entra cuando hay muelle libre
C8	Perecedero	4	5.00	9.00	Entra cuando hay muelle libre

6.2 Resultados por camión Round Robin (Quantum = 2s)

Camión	Prioridad	Burst (s)	T. Espera (s)	T. Retorno (s)	Observación
C2	Perecedero	3	0.00	5.00	Prioridad alta, entra primero
C4	Perecedero	2	0.00	4.00	Prioridad alta, entra primero
C6	Perecedero	1	0.00	3.00	Prioridad alta, entra directo
C8	Perecedero	4	2.00	8.00	Espera quantum, reencolado
C1	Normal	5	3.00	10.00	Cede a perecederos primero
C3	Normal	4	4.00	10.00	Quantum de 2s por turno
C5	Normal	6	5.00	13.00	Mayor espera por quantums
C7	Normal	3	4.00	9.00	Reencolado una vez

6.3 Tabla resumen comparativa

Algoritmo	T. Espera Promedio (s)	T. Retorno Promedio (s)	Observación General
FIFO	2.75	6.25	Efecto convoy en camiones normales
Round Robin	2.25	7.75	Mayor equidad, más cambios de contexto

6.4 Análisis de resultados detallado

Algoritmo FIFO (First-In, First-Out)

El análisis del algoritmo FIFO revela una eficiencia notable en la reducción de la sobrecarga del sistema, ya que, al ser un esquema no expulsivo, los camiones mantienen el uso del muelle hasta completar su carga total. Esto beneficia directamente a los primeros vehículos en ingresar (como C1, C2 y C3), quienes registran tiempos de espera de 0.00s.

Sin embargo, la debilidad estructural de este método es el efecto convoy. En los resultados se observa que la presencia de camiones con ráfagas largas, como C5 (6s), penaliza severamente a los vehículos que llegan después. Un caso crítico es el de C6: a pesar de requerir solo 1s de carga, debe esperar 5s en la cola. Este fenómeno infla el tiempo de espera promedio a 2.75s, demostrando que FIFO no es capaz de gestionar la urgencia de las cargas, tratando por igual a productos normales y perecederos.

Algoritmo Round Robin con Prioridad

El esquema de Round Robin (RR) introduce una gestión más equitativa y dinámica de los recursos. Al implementar un quantum de 2s, el sistema garantiza que ningún camión monopolice un muelle por tiempo prolongado. Lo más destacado es la efectividad de la cola de prioridad: los camiones perecederos (C2, C4, C6) logran tiempos de espera de 0.00s al ser colocados al inicio de la cola, cumpliendo con el objetivo logístico de despachar primero lo más sensible al tiempo.

El costo de esta equidad se refleja en las métricas de retorno. El tiempo de retorno promedio sube a 7.75s (frente a los 6.25s de FIFO). Esto se debe a los cambios de contexto y a la necesidad de reencolado para aquellos camiones cuya carga excede el quantum, como ocurre con C8 o C1. Aunque el sistema es técnicamente más "lento" en promedio, es mucho más predecible y justo para cargas críticas.

Conclusión

Tras evaluar ambos modelos en el simulador, se concluye que la elección del algoritmo debe depender estrictamente de la naturaleza de la carga.

- Eficiencia bruta vs. Calidad de servicio: Si la terminal operara bajo un modelo de carga homogénea donde el tiempo de caducidad no fuera un factor, FIFO sería la opción más eficiente debido a su menor tiempo de retorno total y simplicidad de implementación.
- Optimización logística: No obstante, para un escenario real de gestión de carga mixta como el propuesto, el algoritmo de Round Robin con Prioridad se establece como la mejor estrategia. Aunque genera una mayor latencia promedio, protege el valor económico de los productos perecederos al asegurarles un acceso preferente a los muelles, evitando que queden atrapados detrás de cargas voluminosas, pero no urgentes.

7. Referencias Bibliográficas

- **Silberschatz, A., Galvin, P. B., y Gagne, G.** (2018). *Operating system concepts* (10.ª ed.). Wiley.
- **Tanenbaum, A. S., y Bos, H. (2014).** *Modern operating systems* (4.ª ed.). Pearson.
- **Kerrisk, M. (2010).** *The Linux programming interface: A Linux and UNIX system programming handbook*. No Starch Press.
- **Stallings, W. (2018).** *Operating systems: Internals and design principles* (9.ª ed.). Pearson.
- **Arpaci-Dusseau, R. H., y Arpaci-Dusseau, A. C. (2018).** *Operating systems: Three easy pieces*. Arpaci-Dusseau Books.